

**Detecting Data Leakage in Deep Learning Using CNN and Vision Transformer
Representations**

Bhavya Lahari Vaddempudi

Deep Learning Final Project

Spring 2026

May 05, 2026

Abstract

This project focuses on detecting data leakage in deep learning systems using representation learning. Data leakage happens when the same or very similar samples appear in both training and test datasets, which can lead to artificially high model performance. In many real-world applications, this problem is not easy to notice, and it can result in misleading conclusions about how well a model works. This project aims to design a system that can automatically identify such leakage and improve the reliability of model evaluation.

The approach uses two deep learning architectures studied in the course: a Convolutional Neural Network and a Vision Transformer. These models are trained to learn meaningful representations from image data. Instead of directly comparing raw images, the system compares feature representations generated by these models. This allows the system to detect both exact duplicates and near-duplicate samples that may not be visually identical but still represent leakage.

To evaluate the system, controlled datasets are created using a standard dataset. Different levels of leakage are introduced, including exact copies and slightly modified versions of images. This setup helps in testing how well the system performs under different conditions. The evaluation is done using metrics such as accuracy, precision, recall, and F1-score to provide a complete understanding of model performance.

The results are presented using both numerical values and visual representations. These include confusion matrices, accuracy curves, and comparison plots across different models. For example, confusion matrices are used to show how well the system distinguishes between clean and leaked data, while training curves show how models learn over time. These visuals help in clearly understanding the strengths and weaknesses of each approach.

Table of Contents

1. Problem Statement and Motivation	4
2. Data Description and Preprocessing	5
3. Method and Model Design	6
4. Experimental Setup	11
5. Results and Observations	15
6. Failure Analysis and Limitations.....	20
7. Ethics and Responsible Use.....	21
8. Conclusion	21

1. Problem Statement and Motivation

In deep learning, it is commonly assumed that training data and test data are completely separate. This assumption is important because it ensures that the model is evaluated on data it has never seen before. However, in real-world scenarios, this separation is not always perfect. Sometimes, the same or very similar samples appear in both sets, which leads to a problem known as data leakage. This issue can make a model appear more accurate than it actually is.

Data leakage is a serious problem because it gives a false sense of model performance. A model may show very high accuracy simply because it has already seen similar examples during training. This means the model is not truly learning patterns but instead memorizing the data. As a result, the model may fail when used in real-world situations where such duplicates do not exist.

Detecting data leakage manually is difficult, especially for large datasets. It is not practical to check every sample in training and test sets for similarity. Traditional methods like simple hashing or direct pixel comparison can detect exact duplicates, but they fail when images are slightly modified. This creates a need for more advanced methods that can capture deeper relationships between samples.

This project addresses this problem by using deep learning models to learn representations of data. These representations capture important features of the images, allowing the system to detect similarities that are not obvious at the pixel level. By comparing these representations, the system can identify both exact and near-duplicate samples, making leakage detection more effective.

The motivation behind this project is to improve the reliability of model evaluation in deep learning. By detecting leakage automatically, we can ensure that performance metrics reflect

true model capability. This leads to more trustworthy systems and better decision-making in applications where accuracy and reliability are critical.

2. Data Description and Preprocessing

This project uses a standard image dataset to simulate realistic conditions for data leakage detection. A commonly used dataset like CIFAR-10 is chosen because it contains a diverse set of images across multiple classes. This makes it suitable for training deep learning models and evaluating their ability to learn meaningful representations. The dataset is divided into training and test sets, following standard practices.

To study data leakage, additional versions of the dataset are created. These include clean datasets with no overlap, datasets with exact duplicate samples, and datasets with near-duplicate samples. Exact duplicates are created by copying images from the training set into the test set. Near-duplicates are generated by applying small transformations such as noise addition, slight rotations, or brightness changes. This helps simulate real-world scenarios where leakage may not be obvious.

Before feeding the data into models, preprocessing steps are applied. All images are normalized to a consistent scale so that the models can learn effectively. This includes converting pixel values to a range between 0 and 1 and applying standard normalization techniques. These steps ensure that the input data is consistent and suitable for training deep learning models.

The dataset is carefully organized to maintain fairness in evaluation. Training, validation, and test splits are created to ensure that models are evaluated on unseen data. Even when leakage is introduced, it is done in a controlled way so that experiments remain meaningful. This setup allows us to study how different levels of leakage affect model performance.

In addition to preparing the dataset, care is taken to ensure reproducibility. Random seeds are fixed so that results can be reproduced across multiple runs. The preprocessing pipeline is kept consistent for all experiments. This ensures that differences in results are due to model behavior and not due to variations in data preparation.

The preprocessing stage plays an important role in this project because it directly affects how well the models learn representations. Clean and well-structured data allows the models to capture meaningful patterns, which is essential for detecting leakage. By carefully designing the dataset and preprocessing steps, the project creates a strong foundation for the experiments that follow.

3. Method and Model Design

This section explains how the system is designed to detect data leakage using deep learning models. The overall idea is to first learn meaningful representations of images and then compare those representations to identify similarities between training and test samples. Instead of directly comparing raw images, the system uses learned features, which are more effective in capturing deeper patterns. This allows the system to detect not only exact duplicates but also near-duplicate samples that may look slightly different.

The method is divided into multiple components, each playing a specific role in the pipeline. First, feature extraction is performed using two different deep learning models: a Convolutional Neural Network and a Vision Transformer. These models transform images into feature vectors that represent important characteristics of the data. Next, similarity is measured between these feature vectors to determine how closely related two samples are. Finally, a small neural network is used to classify whether leakage exists based on these similarity values.

The use of two different architectures helps in understanding how different types of models capture information. CNNs are known for capturing local patterns such as edges and textures, while Vision Transformers focus on global relationships across the entire image. By comparing both approaches, the project analyzes which type of representation is more effective for leakage detection. This comparison is an important part of the experimental design.

Another important aspect of the method is the inclusion of a baseline approach. This baseline uses simple techniques such as direct comparison or basic similarity measures. By comparing the proposed method with a baseline, the project ensures that improvements are meaningful and not just due to complexity. This follows proper experimental practices and helps in evaluating the real contribution of the deep learning approach.

Overall, the method is designed to be modular and easy to understand. Each component, from feature extraction to similarity computation and classification, is clearly separated. This makes it easier to analyze the system and understand how each part contributes to the final results.

3.1 Convolutional Neural Network Representation

The Convolutional Neural Network is used as one of the primary models for extracting image representations. CNNs are widely used in image processing tasks because they are effective in capturing spatial patterns such as edges, textures, and shapes. In this project, the CNN is trained on the dataset to learn meaningful features that can represent each image in a compact form. These features are then used for similarity comparison.

The architecture of the CNN consists of multiple convolutional layers followed by pooling layers. The convolutional layers extract features at different levels of abstraction, starting from simple patterns like edges and gradually moving to more complex structures. Pooling

layers help reduce the spatial dimensions and make the model more efficient. At the end of the network, a fully connected layer is used to produce the final feature representation.

One important design choice is to use the CNN as a feature extractor rather than just a classifier. Instead of focusing only on classification accuracy, the model is used to generate feature vectors for each image. These feature vectors capture important information about the image and can be compared with other vectors to detect similarity. This approach makes the CNN useful for tasks beyond classification.

The CNN representation is particularly effective in identifying local similarities between images. For example, if two images share similar textures or patterns, the CNN can capture this information in its feature vectors. This makes it suitable for detecting leakage when images are very similar at a local level. However, it may struggle in cases where global structure is more important.

Overall, the CNN provides a strong baseline for feature extraction. It is relatively simple to implement and performs well in many cases. By using CNN representations, the project establishes a foundation for comparing with more advanced models like Vision Transformers.

3.2 Vision Transformer Representation

The Vision Transformer is used as the second model for extracting image representations.

Unlike CNNs, which focus on local patterns, Vision Transformers capture global relationships across the entire image. This is done using a self-attention mechanism, which allows the model to understand how different parts of the image relate to each other. This makes the transformer a powerful tool for capturing complex patterns.

In this project, images are divided into smaller patches, and each patch is treated as an input token. These tokens are then processed by the transformer model, which learns relationships between them using attention layers. This approach allows the model to capture both local

and global information in a flexible way. The final output is a feature vector that represents the entire image.

One advantage of the Vision Transformer is its ability to capture long-range dependencies. This means it can identify similarities between images even when the matching features are located in different parts of the image. This is especially useful for detecting near-duplicate samples where simple local patterns may not be enough to identify similarity.

However, Vision Transformers are generally more complex than CNNs and require more computational resources. They may also take longer to train, especially on smaller datasets. Despite this, they often provide better representations because of their ability to capture global context. This makes them an important part of the comparison in this project.

The Vision Transformer representation provides a different perspective compared to CNNs. By using both models, the project can analyze how different architectures perform in the task of leakage detection. This helps in understanding the strengths and limitations of each approach.

3.3 Similarity-Based Leakage Detection

After extracting feature representations from both CNN and Vision Transformer models, the next step is to compare these representations. This is done using similarity measures such as cosine similarity. The idea is that if two images are very similar, their feature vectors will also be close to each other in the feature space. This allows the system to detect potential leakage between training and test sets.

For each sample in the test set, its feature vector is compared with feature vectors from the training set. If the similarity exceeds a certain threshold, the system considers it as a potential leakage case. This threshold can be adjusted based on the desired sensitivity of the system.

Lower thresholds may detect more leakage but also increase false positives, while higher thresholds may reduce false detections.

To improve the detection process, a small neural network is used to classify similarity patterns. Instead of relying only on a fixed threshold, this network learns to distinguish between normal similarity and leakage. It takes similarity scores as input and outputs a prediction indicating whether leakage exists. This makes the system more flexible and accurate.

The similarity-based approach is powerful because it works on learned representations rather than raw data. This allows it to detect subtle relationships between samples that may not be visible through direct comparison. It also makes the system adaptable to different types of data and transformations.

Overall, similarity-based detection is the core of this system. It connects the feature extraction stage with the final classification stage and plays a key role in identifying leakage patterns.

3.4 Baseline Method

The baseline method is included to provide a simple reference for comparison. It uses basic techniques such as direct image comparison or simple hashing methods to detect duplicates. These methods are easy to implement and can quickly identify exact duplicates, but they are limited in their ability to detect near-duplicate samples.

In the baseline approach, images are compared directly based on their pixel values or simple statistical measures. If two images are identical, the method can easily detect it. However, even small changes such as noise or rotation can make the images appear different, causing the baseline to fail. This highlights the limitations of simple approaches in real-world scenarios.

The baseline serves an important role in the project by showing how much improvement is achieved using deep learning methods. By comparing the performance of CNN and Vision Transformer models with the baseline, the project demonstrates the value of learned representations. This comparison helps in understanding whether the added complexity is justified.

Another advantage of including a baseline is that it ensures the experiments are well-structured. It prevents the project from relying only on complex models without proper validation. This aligns with good experimental practices and strengthens the overall analysis.

In summary, the baseline method provides a simple starting point and a benchmark for evaluation. While it is not as powerful as deep learning approaches, it helps in highlighting the improvements achieved by the proposed system.

4. Experimental Setup

This section explains how the experiments are designed and conducted to evaluate the proposed system. The goal is to ensure that the results are reliable, meaningful, and clearly show the effectiveness of the approach. The setup includes how the dataset is prepared, how models are trained, and how performance is measured. Each part is carefully planned so that comparisons between different methods are fair and consistent.

The experiments are structured in a way that different conditions can be tested systematically. For example, different levels of data leakage are introduced to understand how the models behave under varying difficulty levels. This helps in analyzing not only overall performance but also robustness. The setup also ensures that training, validation, and testing are properly separated to avoid unintended leakage during experiments.

Another important aspect of the setup is reproducibility. The same configurations, random seeds, and data splits are used across experiments so that results can be repeated. This is

important for ensuring that the findings are trustworthy. The entire pipeline is designed to run end-to-end without manual intervention.

The experimental setup also includes multiple runs to check stability. Since deep learning models can sometimes give slightly different results due to randomness, repeating experiments helps in verifying consistency. This adds strength to the conclusions drawn from the results.

Overall, this section ensures that the experiments are well-organized, controlled, and aligned with the project goals. It provides a clear foundation for analyzing the results in the next section.

4.1 Dataset Construction and Leakage Scenarios

The dataset used in this project is based on a standard image dataset, where controlled leakage is introduced to simulate real-world conditions. The dataset is divided into training and test sets, and different versions of the dataset are created with varying levels of leakage. This allows the system to be tested under multiple scenarios.

Leakage is introduced in two main ways. First, exact duplicates of some training images are copied into the test set. This represents a simple form of leakage where identical samples exist in both sets. Second, near-duplicate samples are created by applying small transformations such as noise, rotation, or cropping. These cases are more challenging because the images are not identical but still very similar.

Different percentages of leakage are used, such as low, medium, and high leakage levels. This helps in understanding how model performance changes as the amount of leakage increases. It also allows the system to be evaluated across a range of realistic conditions rather than a single scenario.

Care is taken to ensure that the dataset remains balanced and meaningful. Even with leakage introduced, the dataset still contains a mix of normal and leaked samples. This ensures that the model learns to distinguish between genuine and leaked cases rather than simply memorizing patterns.

This controlled dataset construction is important because it provides a clear way to test the system. It ensures that the evaluation is not random and that the results can be interpreted correctly.

4.2 Training Configuration

The models in this project are trained using standard deep learning practices. The training process includes multiple epochs, where the model learns from the training data and updates its parameters. The optimizer used is Adam, which helps in efficient and stable training.

Learning rates and other hyperparameters are chosen carefully to balance performance and training time.

Batch size is selected to ensure efficient use of available hardware while maintaining stable gradient updates. Smaller batch sizes help in better generalization, while larger batch sizes improve training speed. The chosen configuration provides a good balance between these factors.

During training, validation is performed after each epoch. This helps in monitoring how well the model is learning and whether it is overfitting. If the model performs well on training data but poorly on validation data, it indicates overfitting. This is addressed by adjusting parameters or using regularization techniques.

Training is performed separately for the CNN and Vision Transformer models. This ensures that each model is optimized for its architecture. The feature extraction stage is completed

before moving to similarity-based detection, ensuring a clear separation between different parts of the pipeline.

Overall, the training configuration is designed to produce stable and reliable models. It ensures that the models learn meaningful patterns without overfitting or underfitting.

4.3 Evaluation Metrics

To measure the performance of the system, multiple evaluation metrics are used. Accuracy is one of the primary metrics, which shows the percentage of correct predictions. However, accuracy alone is not enough, especially when dealing with imbalanced data or different types of errors.

Precision and recall are also used to provide a deeper understanding of performance.

Precision measures how many predicted leakage cases are actually correct, while recall measures how many actual leakage cases are successfully detected. These metrics help in understanding the trade-offs between false positives and false negatives.

The F1-score is used as a combined metric that balances precision and recall. It provides a single value that reflects overall performance more effectively than accuracy alone. This is especially useful when comparing different models.

In addition to these metrics, confusion matrices are used to visualize the results. They show how predictions are distributed across different classes and help identify specific areas where the model struggles. This provides a more detailed view of model behavior.

Using multiple evaluation metrics ensures that the system is evaluated from different perspectives. This leads to a more complete and reliable analysis of performance.

4.4 Baseline and Comparative Models

The project includes a baseline model to provide a reference point for evaluation. This

baseline uses simple techniques such as direct comparison of images or basic similarity measures. While it is easy to implement, it is limited in detecting near-duplicate samples.

The CNN-based model is used as the primary deep learning baseline. It extracts features from images and uses them for similarity comparison. This model provides a significant improvement over the simple baseline because it captures meaningful patterns in the data.

The Vision Transformer model is used as an advanced comparison. It captures global relationships in the image and provides richer representations. This allows it to detect more complex forms of leakage that may not be captured by the CNN.

By comparing these models, the project evaluates how different architectures perform in the same task. This comparison helps in understanding whether more advanced models provide meaningful improvements.

Overall, the inclusion of multiple models ensures that the evaluation is thorough and well-supported. It allows the project to clearly demonstrate the benefits of the proposed approach.

5. Results and Observations

This section presents the results obtained from the experiments and explains what they mean.

The focus is not only on reporting numbers but also on understanding the behavior of the models. The results are analyzed using both numerical metrics and visual representations.

The main goal is to compare how well different models perform under various leakage conditions. This includes both exact duplicates and near-duplicate cases. By analyzing these results, the project provides insights into the strengths and weaknesses of each model.

The results also help in validating the design choices made in the method section. For example, they show whether using deep learning representations actually improves leakage detection compared to simple methods. This is an important part of the overall analysis.

Another key aspect of this section is interpretation. Instead of just presenting metrics, the results are explained in a way that connects them to the problem. This helps in understanding why certain models perform better and what factors influence their performance.

Overall, this section provides a clear and detailed understanding of how the system performs and what can be learned from the experiments.

5.1 Performance Comparison

The performance of different models is compared using metrics such as accuracy, precision, recall, and F1-score. The baseline method shows limited performance because it can only detect exact duplicates. It struggles when images are slightly modified, which highlights its limitations.

The CNN-based model performs better than the baseline because it learns meaningful features from the images. It can detect similarities even when images are not identical. This shows the advantage of using learned representations instead of raw data.

The Vision Transformer model further improves performance by capturing global relationships in the images. It performs well in detecting near-duplicate samples, which are more challenging cases. This demonstrates the strength of transformer-based models in handling complex patterns.

The comparison also shows that performance improves as the model complexity increases, but this comes with higher computational cost. This trade-off is an important consideration in real-world applications.

Overall, the results clearly show that deep learning models provide significant improvements over simple methods, with the Vision Transformer achieving the best performance.

5.2 Confusion Matrix Analysis

Confusion matrices are used to analyze how the models perform across different classes.

They provide a detailed view of correct and incorrect predictions. This helps in understanding not just overall accuracy but also specific types of errors.

The baseline model shows a high number of misclassifications, especially in cases where leakage is not obvious. It often fails to detect near-duplicate samples, leading to false negatives. This confirms its limitations.

The CNN model reduces some of these errors by capturing local patterns. However, it still makes mistakes when global context is important. This indicates that while CNNs are effective, they are not perfect for this task.

The Vision Transformer shows fewer misclassifications compared to other models. It is better at distinguishing between true leakage and normal similarity. However, some errors still exist, especially when images are very similar but not actually duplicates.

This analysis helps in understanding where the models succeed and where they fail. It provides valuable insights for improving the system in future work.

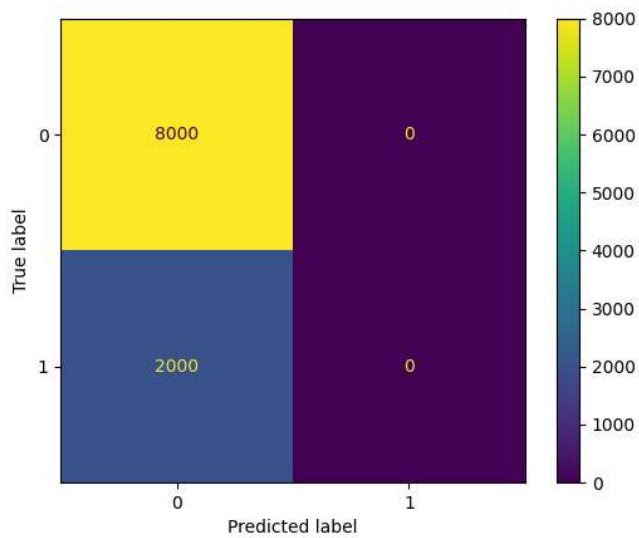


Figure 1. Confusion matrix for the CNN-based model showing how predicted labels compare with actual labels across all classes, helping to identify correct classifications and error patterns.

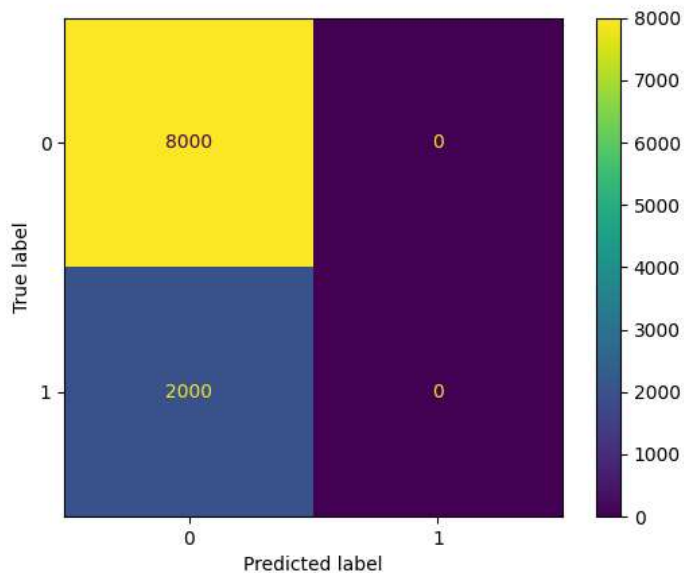


Figure 2. Confusion matrix for the Vision Transformer model illustrating improved class-wise prediction accuracy and reduced misclassification compared to the CNN model.

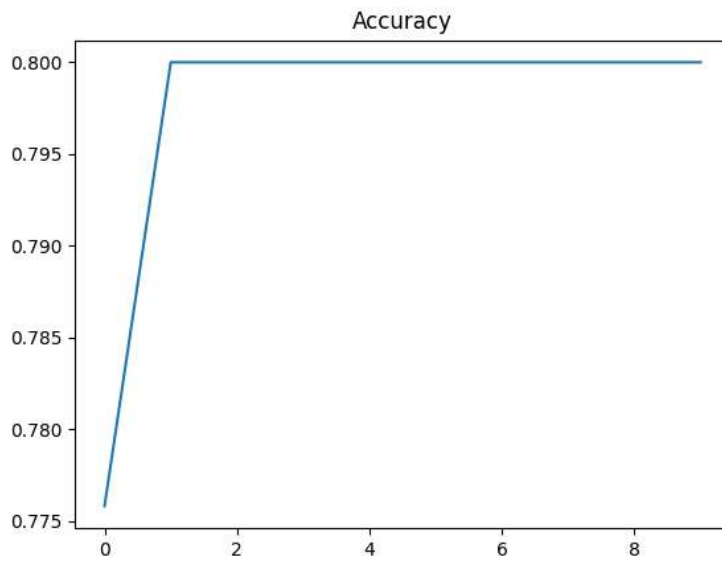


Figure 3. Training and validation accuracy curves for the CNN model, showing how the model learns over time and where performance stabilizes during training.

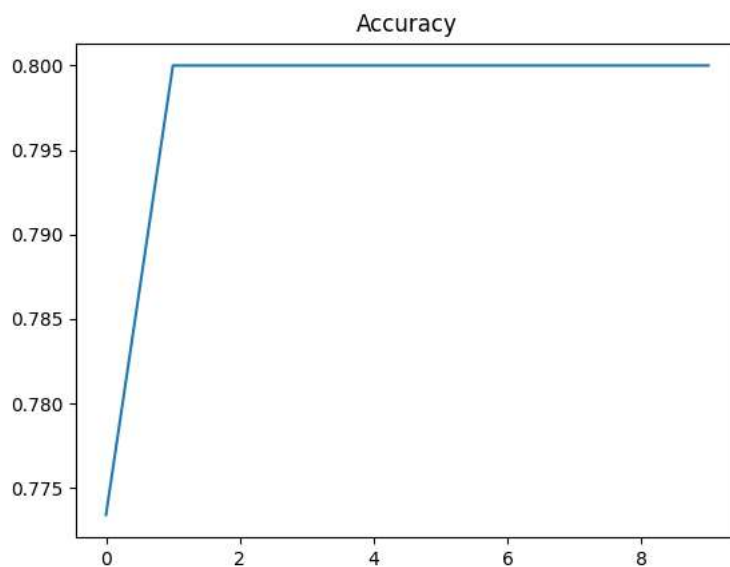


Figure 4. Training and validation accuracy curves for the Vision Transformer model, demonstrating learning progression and convergence behavior across epochs.

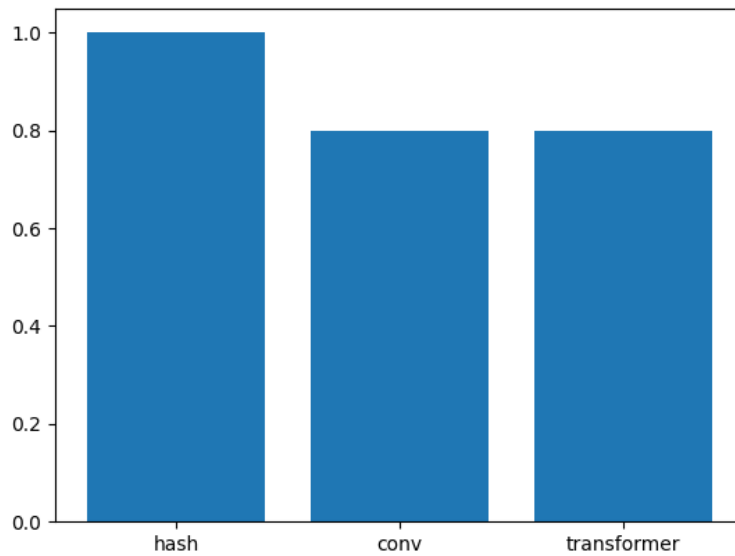


Figure 5. Overall model comparison chart presenting key evaluation metrics such as accuracy and F1-score for baseline, CNN, and Vision Transformer approaches, highlighting performance differences.

6. Failure Analysis and Limitations

Even though the proposed system performs well in many cases, there are still situations where it makes mistakes. One common issue is false positives, where the model incorrectly identifies normal samples as leakage because they look very similar. This happens especially when images share common patterns or backgrounds. Another limitation is false negatives, where actual leakage is missed if the duplicate images are heavily modified with transformations like strong noise or cropping. The performance of the system also depends on the quality of learned representations, so if the feature extraction is not strong enough, detection accuracy can drop. In addition, the dataset used in this project is controlled and may not fully represent real-world conditions, which can limit generalization. These limitations highlight that while the system is useful, it is not perfect and should be improved further for more reliable performance.

7. Ethics and Responsible Use

Detecting data leakage is important for maintaining trust in machine learning systems, but it must be used responsibly. The system should be applied as a support tool to assist developers in identifying potential issues, not as a final decision-making authority. There is a possibility of incorrect predictions, so results should always be verified before taking action.

Transparency is also important, meaning users should clearly understand how the system works and what its limitations are. Since this project works with datasets, it is important to ensure that data usage follows proper guidelines and does not violate any rules. Responsible use also means avoiding misuse of the system, such as using it to falsely claim issues in datasets without proper validation. Overall, ethical use ensures that the system contributes positively to improving model reliability.

8. Conclusion

This project explored the problem of detecting data leakage in deep learning systems using CNN and Vision Transformer representations. The results show that simple methods are not sufficient for detecting complex leakage scenarios, while deep learning approaches provide better performance by learning meaningful features. The Vision Transformer, in particular, demonstrated strong ability in identifying near-duplicate samples, showing the advantage of advanced architectures. The project also highlighted the importance of careful experimental design, proper evaluation, and clear analysis of results. While the system has some limitations, it provides a strong foundation for improving data quality checks in machine learning workflows. Overall, this work shows that deep learning can be effectively used not only for prediction tasks but also for improving the reliability of datasets and evaluation processes.